DOCKER

Theory Notes

Containers • Images • Dockerfile • Deployment

20 Essential Docker Theory Questions & Answers

Prepared by

Mr. P. Sreenivas

Assistant Professor, CSE (AI & ML)

DOCKER — THEORY NOTES

Q1. What is containerization?

Answer

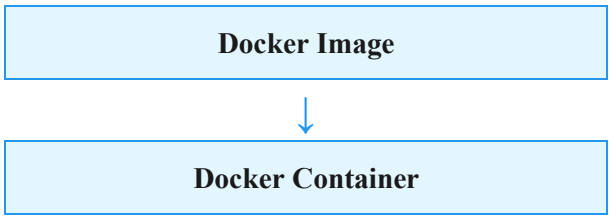
Containerization is a technology that packages an application together with its **code, dependencies, libraries, and configuration** into a standardized unit called a **container**.

It helps applications run consistently across different environments.

Q2. What is a Docker image?

Answer

A Docker image is a **read-only template** used to create Docker containers. It contains the application code, runtime, libraries, dependencies, and required configuration.



Q3. What is a Docker container?

Answer

A Docker container is a **lightweight, isolated environment** created from a Docker image. It contains everything required to run an application.

Docker Image → Docker Container

Q4. What is the difference between a Docker image and a container?

Answer

Docker Image	Docker Container
Read-only template	Running/created instance of an image
Used to create containers	Runs the application
Immutable by design	Has a writable container layer
Can be stored in a registry	Runs on a Docker host

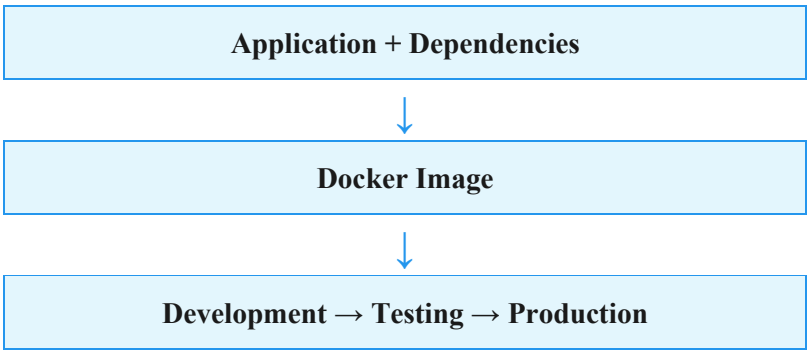
Simple example:

Image = Blueprint
Container = Instance

Q5. How does Docker solve the “works on my machine” problem?

Answer

Docker packages the application together with its required dependencies and environment into an image. The same image can then be used across development, testing, and production environments.



This reduces environment-related differences.

Q6. What are the common causes of the “works on my machine” problem?

Answer

Common causes include:

- Different operating systems
- Different Python/Node.js versions
- Different package versions
- Missing dependencies
- Different environment variables
- Different configurations
- Missing system libraries

> Docker helps standardize the application environment.

Q7. What is a Dockerfile?

Answer

A Dockerfile is a **text file containing instructions used to build a Docker image**.

Example:

```
FROM python:3.11
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

Docker reads these instructions and creates an image.

Q8. What is the purpose of the FROM instruction?

Answer

FROM specifies the **base image** used to build the Docker image.

Example:

```
FROM python:3.11
```

This provides a Python 3.11 environment as the starting point for the image.

Q9. What is the purpose of the WORKDIR instruction?

Answer

WORKDIR sets the **working directory** inside the image/container.

Example:

```
WORKDIR /app
```

Subsequent instructions such as COPY and RUN operate relative to this directory unless another path is specified.

Q10. What is the purpose of the COPY instruction?

Answer

COPY copies files or directories from the Docker **build context** into the image.

Example:

```
COPY . .
```

This commonly copies the application files from the build context into the current working directory in the image.

Q11. What is the purpose of the RUN instruction?

Answer

RUN executes a command **during the Docker image build process**.

Example:

```
RUN pip install -r requirements.txt
```

It can be used to install dependencies or perform setup operations.

Q12. What is the purpose of the CMD instruction?

Answer

CMD specifies the **default command to run when a container starts**.

Example:

```
CMD ["python", "app.py"]
```

It is commonly used to start the application.

Q13. What is the difference between RUN and CMD?

Answer

RUN	CMD
Executes during image building	Defines the default command for container startup
Used for installation/setup	Used to run the application
Creates a new image layer	Provides the default startup command

Example:

```
RUN pip install flask
CMD ["python", "app.py"]
```

Q14. Explain the following Dockerfile line by line.

```
FROM python:3.11
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

Answer

FROM python:3.11 → Uses Python 3.11 as the base image.

WORKDIR /app → Sets /app as the working directory.

COPY requirements.txt . → Copies the requirements file into /app.

RUN pip install -r requirements.txt → Installs Python dependencies while building the image.

COPY . . → Copies application files into /app.

CMD ["python", "app.py"] → Defines the default command that runs when the container starts.

Q15. What is docker build?

Answer

docker build is a command used to **create a Docker image from a Dockerfile**.

Example:

```
docker build -t myapp .
```

Here:

- build → Builds the image
- -t myapp → Assigns a name/tag
- . → Specifies the build context

Q16. What is docker run?

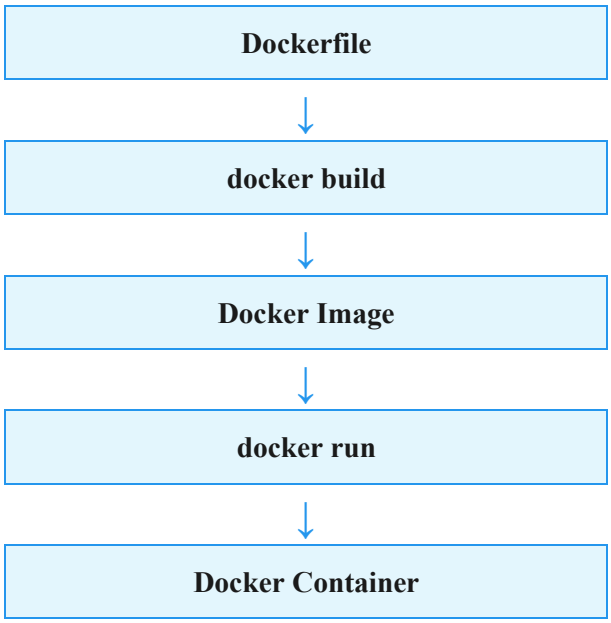
Answer

docker run creates and starts a container from a Docker image.

Example:

```
docker run myapp
```

The basic workflow is:



Q17. What is the difference between local development and production deployment?

Answer

Local Development	Production
Used by developers	Used by real users
Frequent code changes	Stable application version
Debugging commonly enabled	Debugging generally disabled
Development tools may be present	Only required dependencies should be included
Uses development configuration	Uses production configuration

Q18. How are environment-specific configurations handled in Docker?

Answer

Environment-specific configuration can be provided using **environment variables or external configuration** rather than hardcoding values into the Docker image.

Example:

```
docker run -e DATABASE_URL="..." myapp
```

This allows the same image to be used in different environments with different configuration values.

Q19. Why should secrets not be hardcoded in a Dockerfile?

Answer

Secrets such as passwords, API keys, and tokens should not be hardcoded because Dockerfiles and image layers may be stored, shared, or inspected.

> Instead, secrets should be supplied using appropriate secret management or runtime configuration mechanisms.

Q20. Explain the Docker workflow from development to production.

Answer

Step 1 — Development

Create the application and Dockerfile.

Application Code + Dockerfile

Step 2 — Build

Build the Docker image:

```
docker build -t myapp .
```

Step 3 — Test

Run the image as a container:

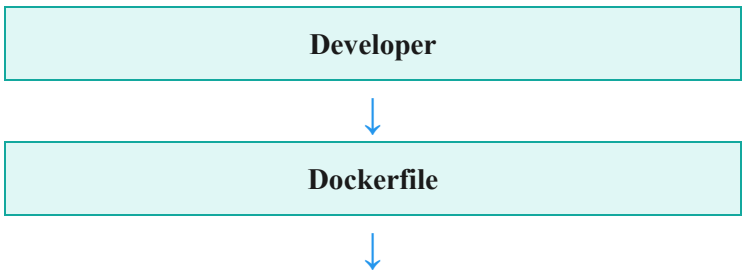
```
docker run myapp
```

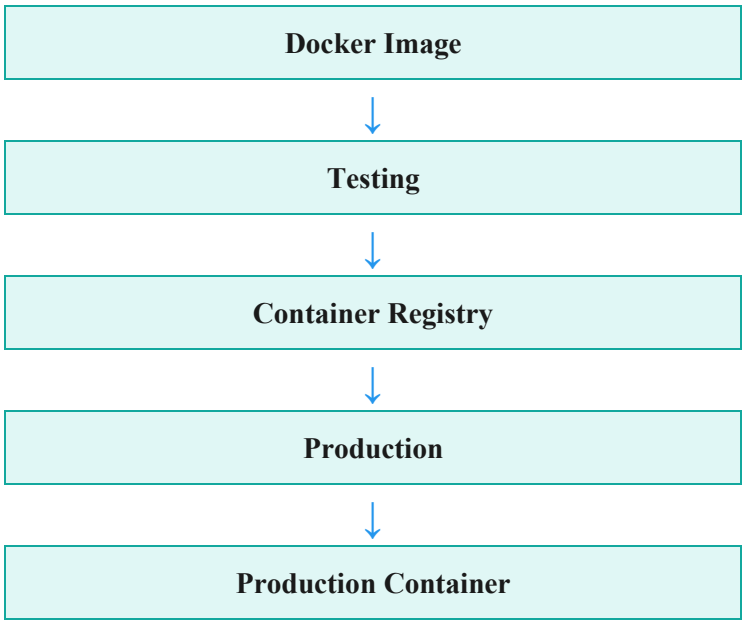
Step 4 — Store/Share

The image can be pushed to a container registry.

Step 5 — Production

The production environment pulls the same image and runs it as a container.





Key idea: Build once, test the same image, and deploy that same image to production.